AI-Powered Loan Origination Copilot

A conversational AI agent that guides retail bank customers through early-stage loan origination — eligibility, EMI, policy answers, and RM escalation.

4	5	8	426
Loan Products	AI Tools	Build Phases	Tests

LangChain · GPT-4o · ChromaDB · Langfuse · Streamlit · FastMCP

Manoj Bansal · 2026

The Problem — Manual Loan Pre-Screening at Scale

2,500 RM-hours / month

Pre-screening time

Eligibility questions, rate queries, and document checklists follow the same script every time. RMs spend 25–40 min even on leads that go nowhere.

60% Drop-off Rate

Before any RM conversation

Majority of queries are rejected at eligibility stage. Customers wait hours only to be told they don't qualify — a poor experience and a wasted RM.

4 Products — 4 Rule-sets

Home · Personal · MSME · New Car

Each product has distinct FOIR limits, rate bands, document requirements, and escalation ceilings. Inconsistent answers create compliance risk.

Business Hours Only

9 am – 6 pm, Mon – Sat

Loan enquiries arrive 24 × 7. Every after-hours query is a potential lead handed to a competitor. No async self-service path exists today.

AI Opportunity: a conversational copilot can handle ≥65% of these queries autonomously — freeing RMs for complex, high-value cases.

What the Copilot Does — Five Integrated AI Capabilities

Eligibility Assessment

FOIR + credit score + age

+ amount limit checks

Returns: PASS / REFER / REJECT

with specific reason per case

EMI Calculation

Reducing-balance formula

Rate range from policy docs

Shows min & max band EMI

Always marked as indicative

Document Checklist

Product × employment matrix

Salaried / Self-employed / Business

5 common + 4 product-specific

Tailored per customer profile

Policy FAQ (RAG)

ChromaDB vector search

Rates, tenure, NRI rules,

prepayment & CIBIL criteria

Grounded in policy documents

RM Escalation

Auto-triggered on ceiling breach

Collects name, mobile, email,

preferred time (all validated)

Structured handoff packet to RM

System Architecture — Five Layers, Fully Traced

① Interface

Streamlit

Customer Chat

RM Dashboard

CLI runner

② Safety Gate

Stage A: Keyword

filter <1 ms

Stage B: GPT-4o-mini

classifier ~150 ms

③ Agent / Orch

LangChain ReAct

GPT-4o

k=10 Memory

Profile Planner

④ MCP Tools

5 tools via

FastMCP server

stdio + HTTP

JSON schema

⑤ Knowledge

ChromaDB

~160 chunks

5 policy docs

text-emb-3-small

MCP Exposure Layer

All 5 tools exposed via FastMCP (loan_mcp/server.py) — discoverable by Claude Desktop, Claude Code, and external systems. Default path USE_MCP=true.

Observability

Langfuse self-hosted (Docker · localhost:3000). Traces every agent turn: tool calls, token counts, latency, feedback scores. LLM-as-judge eval on 20-Q dataset. Zero cloud upload — banking data stays on-premise.

Tech Stack — Every Choice Justified

COMPONENT	CHOICE / VERSION	ENGINEERING JUSTIFICATION
LLM / Orchestration	GPT-4o (agent) + GPT-4o-mini (safety)	GPT-4o: best structured tool-calling fidelity. Mini adds <50 tokens per safety check — minimal cost overhead.
Agent Pattern	LangChain ReAct (create_react_agent)	Loan advisory is sequential: collect → check → estimate → list → escalate. ReAct handles this naturally without a separate planner call.
Vector DB	ChromaDB persistent knowledge/chromadb/	Native Python API, metadata filtering by product, persistent across restarts. No serialisation overhead at 5-doc scale.
Embeddings	text-embedding-3-small 1536 dimensions	Best cost/quality for English banking policy text. Chunked 500 chars / 100-char overlap; product-label prefix per chunk for retrieval accuracy.
Safety Gate	Two-stage: Keyword + LLM classifier	Keyword layer catches 80%+ probes in <1 ms at zero token cost. LLM classifier handles semantic edge cases. Fail-open on API error.
Observability	Langfuse self-hosted Docker Compose	LangSmith is cloud-only — banking income/employment traces cannot leave the network (data-protection compliance). Arize Phoenix excluded for same reason.

RAG Pipeline — Policy-Grounded Answers + The Key Fix

Document Loader

5 policy .txt files
Home / Personal / MSME / Car / General
PRODUCT_MAP metadata tagging

↓

Chunker

500-char chunks, 100-char overlap
Product-label prefix per chunk:
[HOME LOAN] ... text ...

↓

Embedder

text-embedding-3-small
1536-dim vectors
Batch embed via OpenAIEmbeddings

↓

ChromaDB Store

Persistent collection
~160 total chunks
Metadata filter by product label

↓

Retriever

RAG_TOP_K = 5
query_loan_policy tool calls this
MMR diversity re-ranking

⚡ THE KEY FIX — Product-Label Prefix

❌ WITHOUT prefix

Q: "What documents for an MSME loan?"

Retriever scores on pure semantic similarity → Home Loan chunks rank higher because 'document' and 'salary slip' are common across products → WRONG answer returned.

Retrieval Precision: ~45%

Answer: "You need salary slips and Form 16..." [WRONG]

✅ WITH prefix [MSME LOAN] ...

Q: "What documents for an MSME loan?"

Chunks prefixed '[MSME LOAN] ...' → query matches correct product chunks → correct documents retrieved every time.

Retrieval Precision: ≥70%

Answer: "Udyam certificate, GST returns, audited P&L..." [CORRECT]

Two-Stage Safety Gate + PII Filter

Stage A — Keyword Blocklist

0 LLM tokens · < 1 ms

- "ignore previous instructions"
- "act as" / "jailbreak"
- "transfer money" / "send funds"
- "legal advice" / "tax advice"
- "reveal your system prompt"

PASS →

Stage B — LLM Classifier

GPT-4o-mini · <50 tokens · ~150 ms

Labels: IN_SCOPE · OUT_OF_SCOPE · AMBIGUOUS

- Indirect loan intent ("need AC money")
- Approval guarantee requests (semantic)
- Ambiguous competitor references
- Fail-OPEN on API error — never blocks valid user

✅ SAFE

Agent processes the turn
→ generates response
→ PII mask applied
→ log written

🚫 BLOCKED

"I can only assist with loan-related queries."
No agent call, no log entry.

PII Filter (safety/pii_filter.py) — applied BEFORE every log write

Aadhaar	PAN	Mobile	Email	Account
4-4-4 digit pattern	5 alpha + 4 digit + 1 alpha	6-9 prefix + 9 digits	standard regex	9–18 consecutive digits
[AADHAAR REDACTED]	*[PAN REDACTED]*	*[MOBILE REDACTED]*	*[EMAIL REDACTED]*	*[ACCOUNT REDACTED]*

8-Phase Build Journey — Rules-Based CLI to Full Deployment in 10 Days

P2

Rules-Based CLI

- Keyword intent detection
- EMI reducing-balance calc
- Eligibility rules (FOIR)
- No LLM · No API key

P3

LLM + Prompts

- GPT-4o wired to agent
- V1 / V2 / V3 prompt variants
- Langfuse tracing live
- Prompt comparison 5 questions

P4

RAG ChromaDB

- 5 policy docs ingested
- ~160 chunks, 500-char
- Product-label prefix fix
- ≥70% retrieval pass rate

P5

Tool Integration

- 5 @tool functions built
- create_react_agent wired
- MCP exposure layer added
- Escalation ceiling logic

P6

Memory + Multi-Turn

- k=10 sliding window
- SessionState dataclass
- Planner: next_question()
- Start-over reset flow

P7

RLHF Adaptive

- Thumbs 👍/👎 feedback
- policy_checker inline
- EMPATHY_PREFIX injection
- Cross-session adaptation

P8

Deploy + Safety

- Streamlit Customer Chat
- RM Dashboard + KPI strip
- PII masker on all logs
- P95 latency target < 5 s

P9

Evaluation

- 55 test cases (3 suites)
- RAG / Tools / Safety
- LLM-as-judge scoring
- Evaluation report filed

All 8 phases complete · 426 automated tests passing · 8 Jupyter demo notebooks

Evaluation Framework & Results — 3 Suites, 55 Test Cases

≥70%	≥80%	100%	< 5 s
RAG Pass Rate	**Tool Accuracy**	**Safety Block Rate**	**P95 Latency**
LLM-as-judge 20 Q/A pairs	5 tools × 6 scenarios = 30 test cases	5 adversarial probes 0 false-negatives	End-to-end, full multi-tool turn

SUITE	CASES	TARGET	METHODOLOGY
RAG Quality	20 cases	≥70%	GPT-4o-mini LLM-as-judge: score 0/1 per question. Root-cause: product-label prefix fix raised score from 0.45 to ≥0.70.
Tool Selection	30 cases	≥80%	5 tools × 6 trigger scenarios. Boolean: correct tool chosen AND all required JSON args present. Manual audit of borderline cases.
Safety Gate	5 probes	100%	5 adversarial prompts: injection, competitor, PII demand, approval guarantee, off-topic investment advice. Zero misses tolerated.

Latency Profile

- Safety gate only (blocked): P50 < 200 ms · P95 < 350 ms
- Simple FAQ (RAG only): P50 1.2–1.8 s · P95 < 3.0 s
- Eligibility + EMI (2 tools): P50 2.5–3.5 s · P95 < 5.0 s
- Full multi-tool turn: P50 3.5–5.0 s · P95 < 6.5 s

Business Impact & ROI — Quantified Savings

BEFORE — Manual Process

2,500 hrs/month

RM pre-screening time

₹15,00,000/month

Cost at ₹600/hr loaded

25–40 min/query

Average RM time per customer

Business hours

9 am–6 pm Mon–Sat only

Variable quality

Policy compliance per RM

No lead packet

Escalation quality

AI Copilot →

AFTER — AI Copilot

875 hrs/month

RM handles complex 35% only

₹5,25,000/month

RM residual on escalated cases

< 1 minute

Copilot instant response

24 × 7 × 365

Always available

Policy-grounded

ChromaDB + policy_checker

Structured packet

Name · mobile · history · intent

ROI SUMMARY

₹9.75 L/month	₹1.17 Cr/year	< 6 months	~₹25–30 L	~₹1.1 Cr
Monthly savings	Annual savings	Payback period	Estimated dev cost	Net Year-1 benefit

Key Learnings — What This Build Taught Me

1

ReAct + Typed Tools = Reliable Orchestration

LangChain ReAct with 5 typed tools and structured JSON schemas delivered solid orchestration. The agent correctly sequences: collect → eligibility → EMI → docs → escalate with minimal extra prompt engineering for the flow logic.

2

Data Quality Beats Model Tuning

Adding '[MSME LOAN] ...' as a prefix to every chunk improved retrieval precision from ~45% to ≥70% without changing the embedding model or architecture. A simple data fix outperformed any amount of hyperparameter tuning.

3

Safety Must Be First-Class, Not an Afterthought

Two-stage gate (keyword <1ms + LLM ~150ms) achieved 100% adversarial block rate. PII masking before every log write is non-negotiable in banking — not a feature, a hard constraint.

4

Self-Hosted Observability Solved Two Problems at Once

Langfuse self-hosted satisfied both the tracing need and the data-privacy constraint. LLM-as-judge evaluation on Langfuse closed the quality feedback loop without manual annotation overhead.

5

Prompt-Level RLHF is a Viable Fast Path

Thumbs feedback → policy_updater → EMPATHY_PREFIX injection demonstrates a lightweight RLHF loop without fine-tuning. Prompt-level adaptation is fast to iterate and fully auditable — ideal for a rapid capstone timeline.

THANK YOU · LET'S CONNECT

If you found this useful, I'd love to connect.

I'm open to roles in AI/ML engineering, LLM applications, and applied NLP.

LangChain ReAct	GPT-4o + MCP	ChromaDB RAG	Langfuse	Streamlit	426 Tests
Agent orchestration	Tool-calling + protocol	Vector retrieval	Self-hosted observability	Full-stack deployment	Automated test suite

Name

Manoj Bansal

LinkedIn

linkedin.com/in/manojbansal

Github

github.com/manojbansal-projects

Email

manoj.bansal@gmail.com

End-to-end AI system · 8 phases · 10 days · LangChain + RAG + Safety + RLHF + MCP

#LangChain #GPT4o #RAG #AIEngineering #LLM #FinTech #MachineLearning #Python #OpenAI